

Hands-On GNN Chapter 10

A7610 그래프신경망특론 / 조남언

June 10, 2026

Contents

1 링크 예측 문제 개관	1
2 Graph Autoencoder (GAE)	3
3 Variational Graph Autoencoder (VGAE)	4
4 SEAL: 서브그래프 분류로서의 링크 예측	7
5 정리 및 비교	12

목차

Contents

1 링크 예측 문제 개관

Link Prediction이란?

- 그래프 $G = (V, E)$ 에서 관측되지 않은 엣지 $(u, v) \notin E$ 가 실제로 존재할 가능성을 추정
- 입력: 노드 집합 V , 현재까지 관측된 엣지 E_{obs} , (선택) 노드 특징 X
- 출력: 임의의 노드 쌍에 대한 연결 점수 $s(u, v) \in [0, 1]$

대표 응용

- 소셜 네트워크의 친구 추천
- 추천 시스템(사용자-아이템 이분그래프)
- 단백질 상호작용 예측, 약물-표적 상호작용
- 지식 그래프 자동 완성

암묵적 전제: 지금 보이는 그래프는 표본이다

- 참 엣지 집합 E^* 가 있고, 관측 $E_{\text{obs}} \subset E^*$ 는 부분 관측 — population은 영원히 모르고 표본만 손에 있다
- “관측 안 됨”의 두 의미(수학적으로론 같은 쌍 이진 분류로 환원):
 - 결측 설정: 참 그래프는 이미 존재, 관측이 불완전 (단백질 상호작용, KG 완성)

- 미래 설정: 그래프가 자라는 중 (친구 추천) — population = 미래의 그래프
- 노드 집합 V 는 완전 관측 전제 (transductive) — 새 노드 등장은 별도 설정 (→ SEAL 복선)
- 벤치마크의 뒤틀림: 정답 그래프를 알면서 RandomLinkSplit으로 일부러 숨김 = **MCAR**(Missing Completely At Random) 가정. 현실 결측은 대개 **MNAR**(Missing Not At Random — 결측이 특정 영역에 몰림) → 벤치마크가 실전을 과대평가할 수 있음
- 음성 라벨도 사실 없음: 비관측 쌍 = 진짜 비엣지 or 결측 엣지 — **PU-learning**(Positive-Unlabeled) 구조

전통적 휴리스틱과 학습 기반의 차이

표기: u, v = 후보 간선의 양 끝 노드, $\Gamma(u) = u$ 의 이웃 집합 (감마 함수 아님!), w = 공통 이웃 순회 변수

접근	점수 함수 $s(u, v)$
Common Neighbors	$ \Gamma(u) \cap \Gamma(v) $
Jaccard	$ \Gamma(u) \cap \Gamma(v) / \Gamma(u) \cup \Gamma(v) $
Adamic-Adar	$\sum_{w \in \Gamma(u) \cap \Gamma(v)} 1 / \log \Gamma(w) $
Katz, PageRank	경로 기반 / 랜덤워크 기반
GAE/VGAE	$\sigma(\mathbf{z}_u^\top \mathbf{z}_v)$, $\mathbf{z} = \text{GNN}(X, A)$
SEAL	두 노드의 enclosing subgraph를 통째로 분류

- Adamic-Adar의 직관: 공통 이웃 w 가 희귀할수록 강한 증거 — “동호회 멤버 공통지인 > 연예인 공통지인”. 그래서 $1 / \log |\Gamma(w)|$ 로 기여를 줌
- 휴리스틱: 도메인마다 잘 통하는 지표가 다름 (정해진 가정 위에서만 작동)
- GNN 기반: 데이터로부터 적절한 표현/점수 함수를 학습

Chapter 10에서 다루는 두 가지 모델

- **VGAE** (Variational Graph Autoencoder, Kipf & Welling, 2016)
 - 노드 임베딩을 확률분포로 학습
 - 디코더는 단순 내적 — 전체 그래프 수준에서 한 번에 학습
- **SEAL** (Zhang & Chen, 2018)
 - 각 후보 링크 주변의 enclosing subgraph를 추출
 - DRNL로 노드 라벨을 부여한 뒤 서브그래프 분류 문제로 풀이

한 줄 비교: VGAE는 한 그래프 = 한 예제, SEAL은 한 후보 링크 = 한 예제.

데이터 분할: RandomLinkSplit

- 노드 분류와 달리, 링크 예측은 엣지를 train/val/test로 나눔
- 음성 엣지(negative edge)를 무작위로 샘플링해 이진 분류 데이터 구성

```
from torch_geometric.transforms import RandomLinkSplit
transform = RandomLinkSplit(
    num_val=0.05, num_test=0.1,
    is_undirected=True, split_labels=True,
    add_negative_train_samples=False, # 학습은 매 epoch 새로 샘플링
)
```

- test/val 엿지는 메시지 패싱 그래프에서도 빠진다 (누설 방지)
- 음성은 “전수 라벨링”이 아니라 샘플링: Cora만 해도 가능한 쌍 366만 vs 엿지 1만
 - val/test 음성: 양성과 같은 수로 뽑아 고정 (평가 재현성)
 - train 음성: 매 epoch `recon_loss` 안에서 재샘플링 (데이터 증강 효과)
- “없는 엿지의 바다에서 양성과 같은 수만큼 떠서 음성 대표로 쓴다”

평가지표: AUC와 AP

왜 순위 기반 지표인가 — 확정 음성 라벨이 없고 (PU 구조), 임계값은 응용마다 다르다.

- **AUC** (Area Under the ROC Curve) = $P(s(\text{무작위 양성}) > s(\text{무작위 음성}))$ — 양성·음성 쌍 대결에서 양성인 이길 확률. 0.5 = 무작위, 1.0 = 완벽한 분리
- **AP** (Average Precision) = 점수 내림차순으로 읽어내려가며 양성을 만날 때마다 그 시점의 precision을 평균 — 순위 상위권 오염에 민감

손계산 예시 (양성 3, 음성 3) — 점수순: P(.9), N(.7), P(.6), P(.4), N(.3), N(.2)

- AUC: 양성×음성 9쌍 중 양성인 이기는 쌍 7개 → $7/9 \approx 0.78$
- AP: 양성 위치 1, 3, 4등 → precision $\frac{1}{1}, \frac{2}{3}, \frac{3}{4}$ → 평균 ≈ 0.81

1:1 균형 평가셋(이 장 설정)에선 둘이 비슷 (본 실험 0.87/0.87). 음성이 압도적인 실전 top- k 추천에선 $AP \cdot \text{precision}@k$ 가 실용 가치에 더 정렬된 지표.

2 Graph Autoencoder (GAE)

인코더-디코더의 두 혈통: GAE는 어느 쪽인가

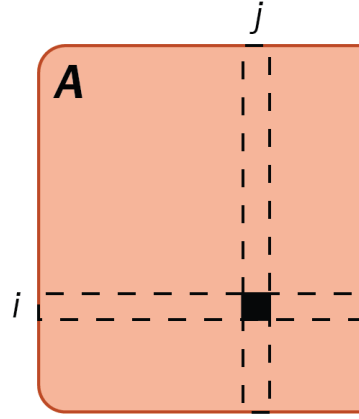
	오토인코더 갈래	seq2seq 갈래
디코더 출력 라벨	입력 자기 자신	다른 대상 (번역문 등)
목적	불필요 (입력 = 정답)	필요 (병렬 코퍼스 등)
대표	표현 학습·압축·생성	변환 (transduction)
	AE, DAE, VAE, GAE/VGAE	RNN seq2seq, Transformer

- **GAE = VAE**의 그래프 이식 (Kipf & Welling, 2016). 이식에서 바뀐 두 지점:
 1. 복원 대상이 인접행렬 A 만 — 입력은 (X, A) 인데 피쳐 X 는 재료로만 씀
 2. 디코더가 학습 없는 내적 — VAE 디코더는 보통 신경망
- 압축이 곧 예측: 병목 Z ($N \times 16$)는 A ($N \times N$)를 통째로 외울 수 없음 → 연결의 규칙성을 학습 → “규칙상 있어야 할 자리”에 높은 점수 = 관측 안 된 쌍의 예측

GAE의 아이디어

- 인코더-디코더 구조를 그래프에 그대로 이식
- 인코더: $\mathbf{Z} = \text{GCN}(X, A) \in \mathbb{R}^{N \times d}$
- 디코더: 노드 임베딩의 내적 → 인접행렬 복원

$$\hat{A} = \sigma(\mathbf{ZZ}^\top)$$



$\hat{A}_{ij} = \sigma(\mathbf{z}_i^\top \mathbf{z}_j)$ 로 임의의 쌍을 한 번에 점수화. 유사한 노드끼리 임베딩 공간에서 가까워지도록 학습.
(Labonne, 2023)

GAE의 재구성 손실

이진 교차엔트로피 형태:

$$\mathcal{L}_{\text{recon}} = - \sum_{(u,v) \in E^+} \log \hat{A}_{uv} - \sum_{(u,v) \in E^-} \log(1 - \hat{A}_{uv})$$

- E^+ : 양성 엣지, E^- : 무작위 음성 엣지
- 클래스 불균형(양성 $\ll$ 음성) 때문에 양/음 같은 수로 샘플링하는 것이 일반적
- 단점: 임베딩이 결정적(deterministic) — 노드별 불확실성을 표현 못함

내적 디코더가 강제하는 가정

$$\hat{A}_{uv} = \sigma(\mathbf{z}_u^\top \mathbf{z}_v)$$

이 한 줄 = **homophily** 가설: “임베딩 공간에서 가까우면 연결된다”

- 디코더가 단순한 게 아니라, 디코더의 형태가 강한 가정을 강제한다

가정이 깨지는 곳

- **Heterophily**: 이질적인 노드끼리 연결 (사기범-정상 사용자 엣지, 상보적 단백질 결합)
- 방향 그래프: $\sigma(\mathbf{z}_u^\top \mathbf{z}_v) = \sigma(\mathbf{z}_v^\top \mathbf{z}_u)$ — 항상 대칭이라 표현 불가
- 글로벌 구조: 내적은 두 점 사이 관계만 — 더 큰 구조 패턴은 못 봄

Cora(인용 네트워크)는 homophily가 강함 $\rightarrow$ VGAE에 유리한 무대

3 Variational Graph Autoencoder (VGAE)

VGAE: 확률적 인코더

각 노드 i 의 잠재변수 $\mathbf{z}_i$ 가 가우시안을 따른다고 가정:

$$q(\mathbf{z}_i | X, A) = \mathcal{N}(\boldsymbol{\mu}_i, \text{diag}(\boldsymbol{\sigma}_i^2))$$

두 개의 GCN으로 평균과 로그분산을 동시에 출력:

$$\boldsymbol{\mu} = \text{GCN}_{\boldsymbol{\mu}}(X, A)$$

$$\log \boldsymbol{\sigma} = \text{GCN}_{\boldsymbol{\sigma}}(X, A)$$

재매개화(reparameterization): $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$

생성 모델 관점: $A_{uv} \sim \text{Bernoulli}(\sigma(\mathbf{z}_u^\top \mathbf{z}_v))$ — 관측 그래프는 이 모델에서 뽑힌 하나의 표본. “그래프는 표본이다”
프레이밍의 형식화이며, 그래서 변분추론(V)이 자연스럽게 붙는다.

VGAE 손실

$$\mathcal{L}_{\text{VGAE}} = \underbrace{\mathcal{L}_{\text{recon}}}_{\text{엣지 복원}} + \frac{1}{N} \underbrace{\text{KL}[q(Z | X, A) \| p(Z)]}_{\text{사전분포로의 정규화}}$$

- 사전분포 $p(Z) = \mathcal{N}(0, I)$
- KL은 정규화 역할 — 임베딩이 너무 자유로워지지 않게 함
- $1/N$ 가중치는 노드 수에 비례한 KL을 노드당 평균으로 정규화하는 효과
- 디코더는 GAE와 동일: $\hat{A}_{uv} = \sigma(\mathbf{z}_u^\top \mathbf{z}_v)$

PyG의 VGAE 인코더 구현

```
from torch_geometric.nn import GCNConv, VGAE

class Encoder(torch.nn.Module):
    def __init__(self, dim_in, dim_out):
        super().__init__()
        self.conv1 = GCNConv(dim_in, 2 * dim_out)
        self.conv_mu = GCNConv(2 * dim_out, dim_out)
        self.conv_logstd = GCNConv(2 * dim_out, dim_out)

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index).relu()
        return (self.conv_mu(x, edge_index),
                self.conv_logstd(x, edge_index))

model = VGAE(Encoder(dataset.num_features, 16)).to(device)
```

- VGAE 래퍼가 reparameterization, recon_loss, kl_loss 제공

VGAE 학습 루프

```
def train():
    model.train()
    optimizer.zero_grad()
    z = model.encode(train_data.x, train_data.edge_index)
    loss = model.recon_loss(z, train_data.pos_edge_label_index) \
        + (1 / train_data.num_nodes) * model.kl_loss()
    loss.backward(); optimizer.step()
    return float(loss)

@torch.no_grad()
def test(data):
    model.eval()
    z = model.encode(data.x, data.edge_index)
    return model.test(z, data.pos_edge_label_index,
                      data.neg_edge_label_index)
```

- recon_loss는 내부적으로 음성 엣지를 매 epoch 자동 샘플링
- test 결과: AUC, AP

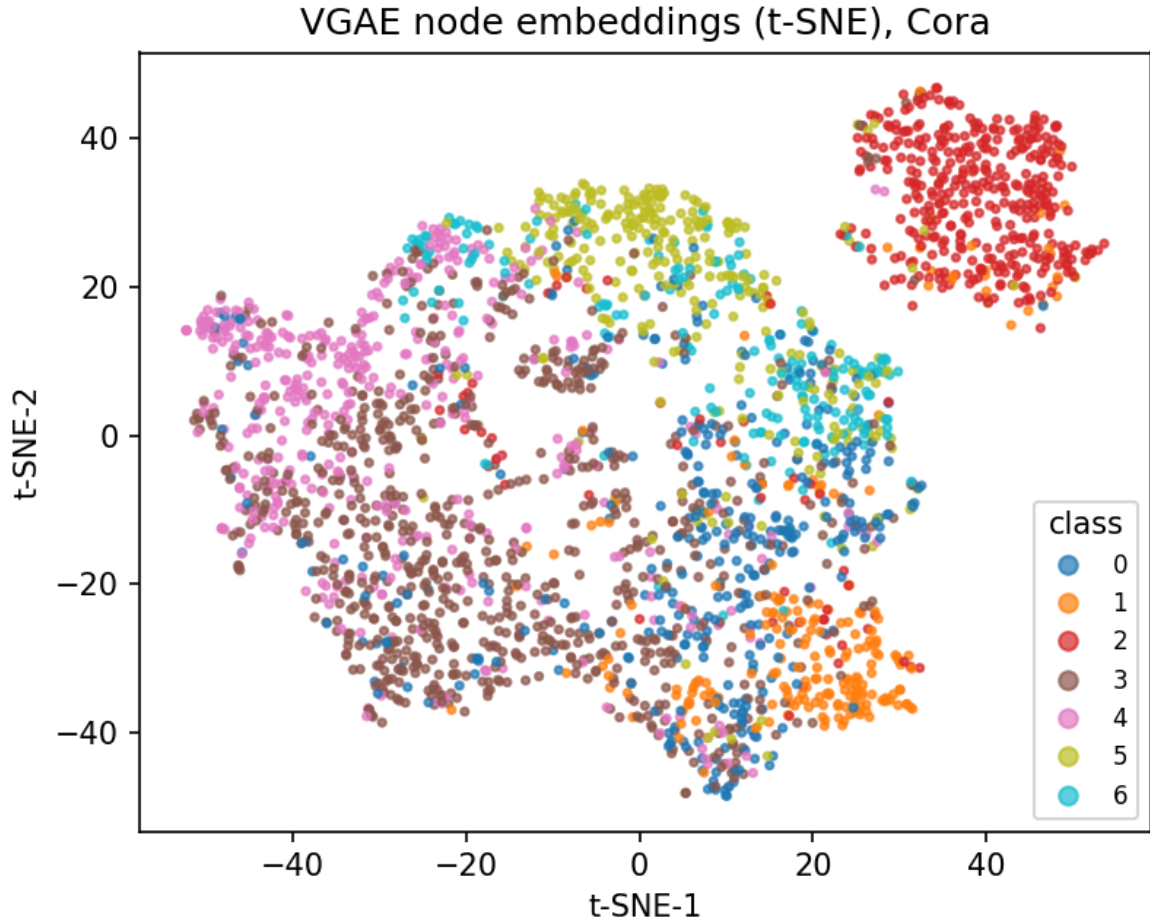
Cora 데이터셋 결과

- Encoder: GCN(1433→32) → GCN(32→16)×2
- 300 epoch, Adam (lr=0.01)
- 책: Test AUC ≈ 0.88 , AP ≈ 0.90
- 본 실험 (CPU, 시드=0): **AUC = 0.869, AP = 0.872**



$\hat{A}$ 의 블록 구조 $\rightarrow$ 같은 클래스 노드끼리 높은 점수가 매겨짐 (homophily 학습됨).

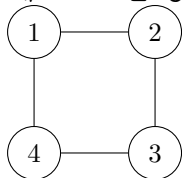
VGAE 임베딩 시각화 (t-SNE)



- 16차원 잠재공간 $\rightarrow$ t-SNE 2D 투영
- Cora의 7개 클래스가 무지도 학습만으로 어느 정도 군집화됨
- 링크 예측만 학습했는데 클래스 구조가 자연스럽게 드러남 — homophily 그래프에서 링크와 클래스는 강하게 결합
- 단점도 같은 그림에서: 일부 클래스(0, 3)는 임베딩이 흐트러져 있음

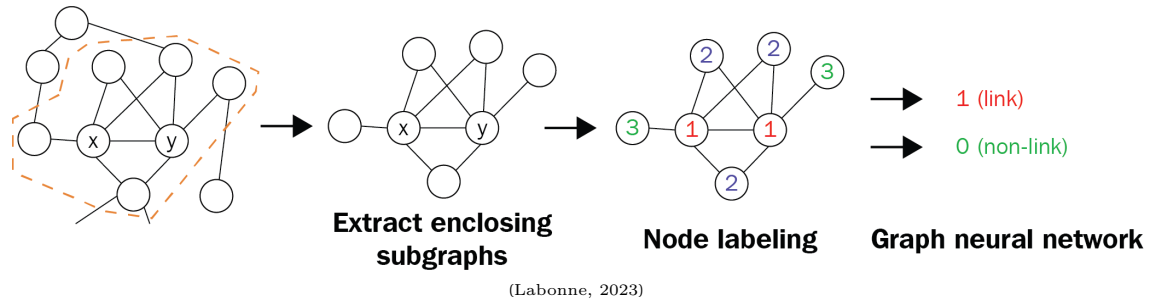
4 SEAL: 서브그래프 분류로서의 링크 예측

왜 GNN은 링크 예측이 어려운가: 4-cycle 반례



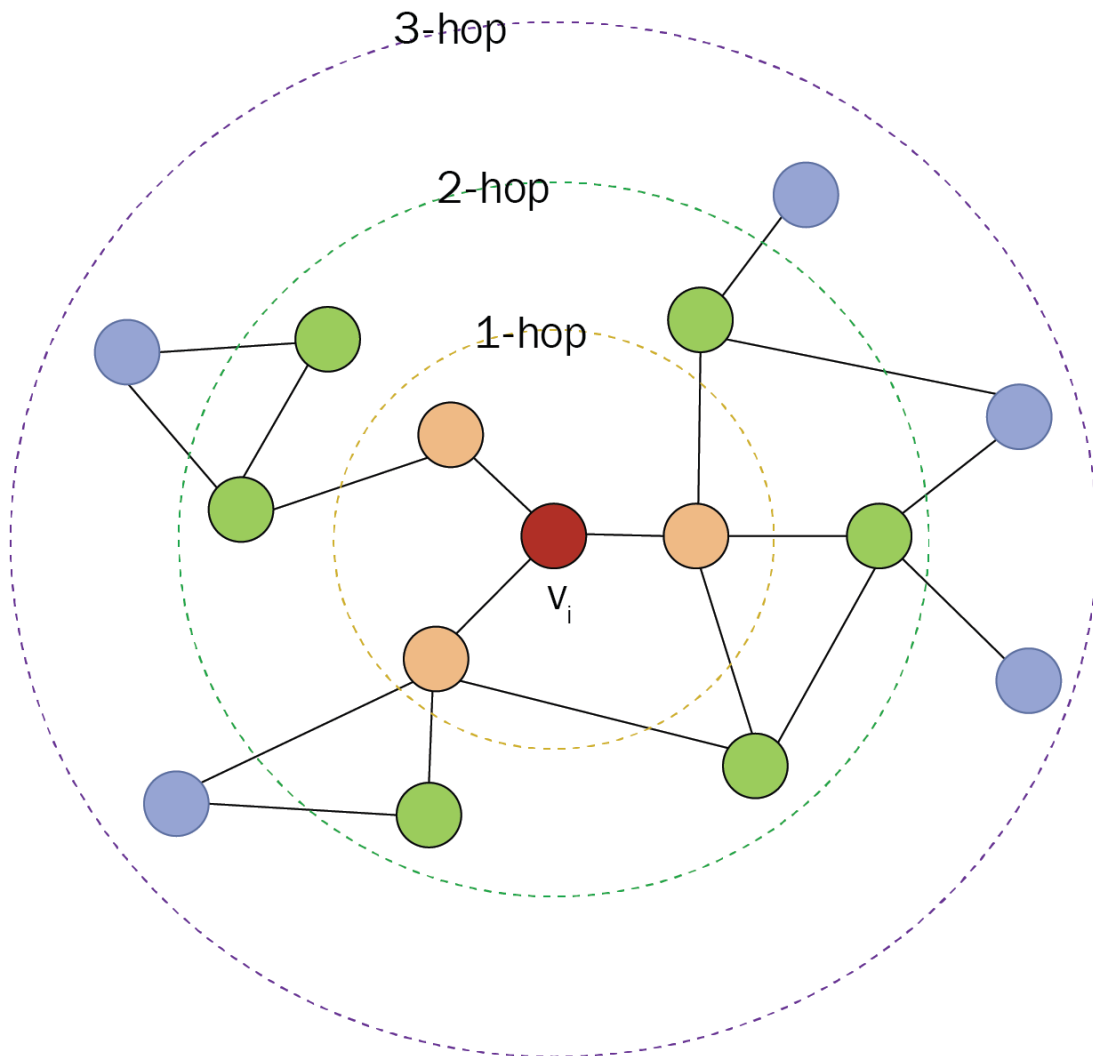
- 모든 노드 차수 2, 이웃 모양 동일 — feature까지 같으면 GNN은 $\mathbf{z}_1 = \mathbf{z}_2 = \mathbf{z}_3 = \mathbf{z}_4$
- 내적 디코더: $\sigma(\mathbf{z}_1^\top \mathbf{z}_2) = \sigma(\mathbf{z}_1^\top \mathbf{z}_3)$ — 그런데 (1,2)는 엣지, (1,3)은 비엣지. 구분 불가
- 학습 부족이 아니라 1-WL 표현력의 구조적 한계 (Xu et al., 2019) — 1-WL이 같은 색을 주는 노드는 GNN도 같은 임베딩을 줌
- 처방: 노드 feature에 “타깃이 누구인지”를 박아넣기 = *labeling trick* $\rightarrow$ SEAL의 DRNL

SEAL의 출발점



- 핵심 통찰: 링크의 존재 여부는 두 노드 주변의 작은 구조에 의해 강하게 결정된다.
- 휴리스틱(CN, Adamic-Adar, Katz)들이 모두 **enclosing subgraph**의 함수
- 링크 예측을 이진 그래프 분류로 환원: 입력 = 후보 (u, v) 주변 k -hop, 출력 = 링크 1/0

k -hop 이웃 개념



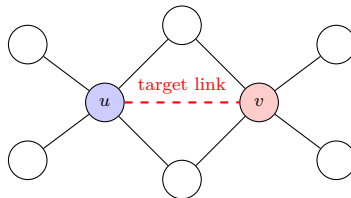
(Labonne,

2023)

- k -hop 이웃: 중심 노드로부터 최대 k 개의 엣지로 도달 가능한 노드 집합
- SEAL은 두 타깃 노드의 k -hop 이웃 합집합을 enclosing subgraph로 사용
- $k = 2$ 가 표준 — 너무 크면 비용 폭발, 너무 작으면 정보 부족

Enclosing Subgraph 추출

- 후보 노드 쌍 (u, v) 에 대해 두 노드의 k -hop 이웃 합집합으로 서브그래프 구성
- PyG의 `k_hop_subgraph({u,v}, k=2, ..., relabel_nodes=True)`



- 타겟 링크 (u, v) 자체는 서브그래프에서 제거 (정답을 보고 학습하면 안 됨)
- 학습/평가는 추출된 서브그래프 전체의 구조와 노드 정보로 결정

타겟 엣지 제거 코드

```
sub_nodes, sub_edge_index, mapping, _ = k_hop_subgraph(
    [src, dst], 2, dataset.edge_index, relabel_nodes=True)
src, dst = mapping.tolist()

# Remove target link from the subgraph
mask1 = (sub_edge_index[0] != src) | (sub_edge_index[1] != dst)
mask2 = (sub_edge_index[0] != dst) | (sub_edge_index[1] != src)
sub_edge_index = sub_edge_index[:, mask1 & mask2]
```

- 양방향 모두 제거 (`is_undirected=True` 가정)
- 양성/음성 쌍 모두 동일한 절차로 처리해야 데이터 분포가 일치

DRNL: Double-Radius Node Labeling

타겟 두 노드까지의 거리로 각 노드에 라벨을 부여:

- $d_x = d(x, u), d_y = d(x, v)$
- 평균 $d = (d_x + d_y)/2$, 모듈로 $r = (d_x + d_y) \bmod 2$

$$z(x) = 1 + \min(d_x, d_y) + d \cdot (d + r - 1)$$

타겟 노드 두 개는 라벨 1 고정, 도달 불가 노드는 0. 왜 이렇게?

- GNN은 노드 자체에 누가 타겟인지 모름 — 라벨로 명시적으로 알려줘야 함
- 두 거리를 따로 쓰지 않고 하나의 정수로 인코딩 → one-hot 임베딩이 깔끔
- 거리 정보로 휴리스틱(CN, AA, ...)의 표현력을 포함

DRNL 공식 음미: 좌표 두 개를 정수 하나로 접기

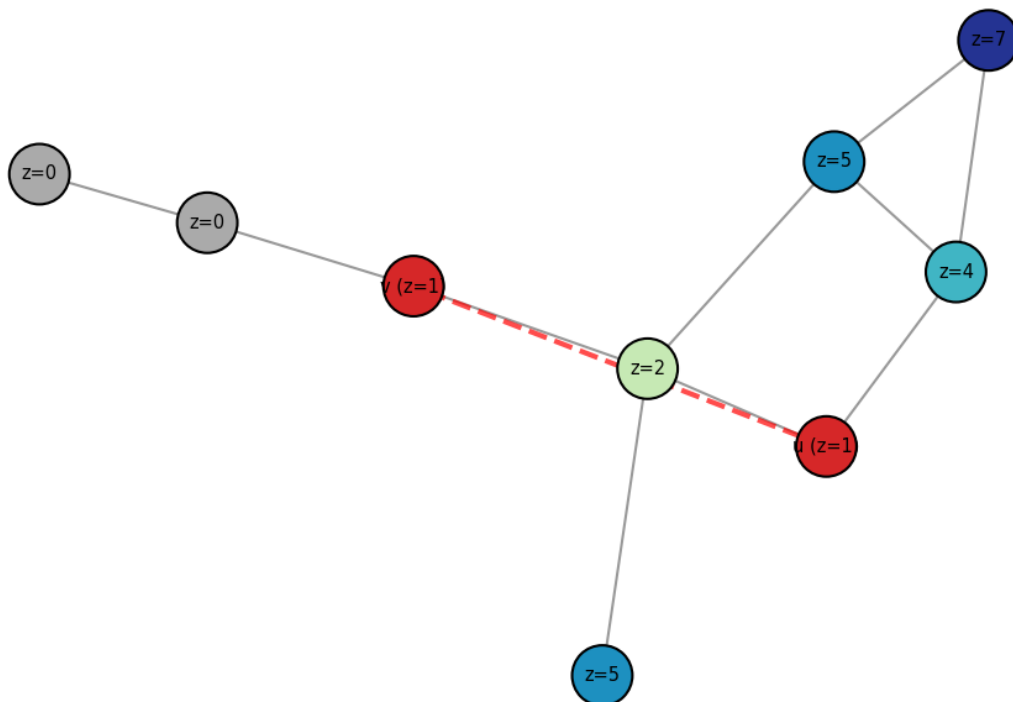
각 노드는 타겟 쌍에 대한 좌표 (d_x, d_y) 를 가짐 — DRNL은 이를 충돌 없이 정수 하나로 접는 perfect

	(d_x, d_y)	$d_x + d_y$	z	의미
hash:	타겟	—	1	u, v 본인
	(1, 1)	2	2	공통 이웃
	(1, 2)	3	3	
	(1, 3)	4	4	
	(2, 2)	4	5	
	(2, 3)	5	7	

- 마지막 항 $d(d+r-1) = \text{“합이 더 작은 좌표들이 이미 차지한 칸 수”}$ (블록 오프셋), $\min = \text{블록 내 순번}$ — *Cantor pairing*식 대각선 배정
- 단사·대칭·단조: 정보 손실 없음 / $(1,3)$ 과 $(3,1)$ 은 같은 z / 멀수록 z 큼
- 공식 자체는 아무래도 좋다 — 증명에 쓰이는 성질은 단사성뿐, 닫힌 형식은 $O(1)$ 계산용. 진짜 발명은 라벨이 타깃 쌍에 의존한다는 발상 (labeling trick)
- $z=2$ 노드 개수 세기 = CN 휴리스틱, $z=2$ 노드의 차수까지 보면 AA — 휴리스틱이 라벨의 함수로 자동 포섭됨

DRNL 라벨링 예시 (실제 Cora 서브그래프)

DRNL labels on a Cora enclosing subgraph ($k=2, N=9$)
red dashed = target link removed during training



- 빨간 노드 = 타깃 (u, v) , 점선 빨간 엣지 = 제거된 타깃 엣지
- 가운데 $z=2$ 노드는 두 타깃의 공통 이웃 (Common Neighbor) — 휴리스틱이 잡아내는 신호
- $z=4, 5, 7$ 로 갈수록 타깃에서 먼 노드. $z=0$ 노드들은 src 제거 시 dst 도달 불가

SEAL Cora 실행 결과 (본 실행)

- 30 epoch, Adam ($\text{lr}=10^{-4}$), BCEWithLogitsLoss
- 책 결과: $\text{AUC} \approx 0.89$, $\text{AP} \approx 0.90$
- 본 실행 (CPU, 시드=0): **$\text{AUC} = 0.786$, $\text{AP} = 0.802$**

차이 원인 추정

- Epoch 수가 30회로 짧음 — DGCNN은 더 긴 학습에서 안정화됨
- BCEWithLogitsLoss를 sigmoid 출력에 사용 (책 코드의 알려진 이슈) — 학습 신호가 약함
- 음성 샘플링 매번 무작위라 시드 영향

DRNL 라벨 계산 (책 코드 발췌)

```
# d_src : 노드 -> src 거리 (dst 그래프에서 잠시 제외하고 계산)
# d_dst : 노드 -> dst 거리 (src 잠시 제외하고 계산)
dist = d_src + d_dst
z = 1 + torch.min(d_src, d_dst) \
    + (dist // 2) * (dist // 2 + (dist % 2) - 1)
z[src], z[dst] = 1., 1.
z[torch.isnan(z)] = 0.

node_labels = F.one_hot(z.to(torch.long), num_classes=200).to(torch.float)
node_emb = node_features # 선택() 원본 노드 feature 결합
node_x = torch.cat([node_emb, node_labels], dim=1)
data = Data(x=node_x, z=z, edge_index=sub_edge_index, y=y)
```

- `shortest_path(..., indices=src)`로 거리 계산
- `dst`를 빼고 `d_src`를, `src`를 빼고 `d_dst`를 — 경로 누설 방지

DGCNN(Deep Graph CNN): SEAL의 분류기

4-layer GCN → SortPooling → 1D CNN → Dense

- GCN layers: $32 \rightarrow 32 \rightarrow 32 \rightarrow 1$ 채널, `tanh` 활성화, 모두 `concat`
- SortPooling: 마지막 채널 기준으로 상위 $k = 30$ 노드만 정렬해 고정 길이로
- Conv1D: 정렬된 시퀀스를 1D 합성곱으로 처리 → 텍스트 분류 형태
- Dense → Sigmoid: 링크 존재 확률

서브그래프 크기가 매번 달라도 고정 차원 표현을 얻는 것이 SortPooling의 역할.

DGCNN 코드 (요지)

```
class DGCNN(torch.nn.Module):
    def __init__(self, dim_in, k=30):
        super().__init__()
        self.gcn1 = GCNConv(dim_in, 32)
        self.gcn2 = GCNConv(32, 32)
        self.gcn3 = GCNConv(32, 32)
        self.gcn4 = GCNConv(32, 1)
        self.global_pool = aggr.SortAggregation(k=k)
        self.conv1 = Conv1d(1, 16, kernel_size=97, stride=97)
        self.conv2 = Conv1d(16, 32, kernel_size=5, stride=1)
        self.maxpool = MaxPool1d(2, 2)
        self.linear1 = Linear(352, 128); self.dropout = Dropout(0.5)
        self.linear2 = Linear(128, 1)
```

- GCN 출력을 *layer-wise concat* 한 뒤 SortPool로 $k = 30$ 개 노드 선택

트레이드오프

SEAL의 장점

- 표현력이 강함 — DRNL이 거리/구조 정보를 명시적으로 인코딩, 휴리스틱들을 일반화
- 인덕티브(inductive): 학습된 모델로 새 그래프에서도 추론 가능

SEAL의 단점

- 후보 링크 하나당 서브그래프 추출 + DRNL 전처리가 필요해 비용이 큼
- VGAE는 한 번의 *encode* 후 모든 쌍에 대해 점수를 즉시 계산 가능
- 대규모 그래프(수억 노드)에선 사실상 적용 불가

5 정리 및 비교

점수함수의 형태 자체가 다르다

GAE/VGAE — node-centric: $\text{score}(u, v) = \sigma(\mathbf{z}_u^\top \mathbf{z}_v)$

- 점수함수에 학습 파라미터가 없다 — 학습은 전부 인코더에서, 디코딩은 고정 규칙
- $\mathbf{z}_u$ 는 한 번 계산되어 어떤 상대와도 재사용 — 쌍의 관계 구조를 직접 보지 않음

SEAL — subgraph-centric: $\text{score}(u, v) = \text{GNN}_\theta(\text{subgraph}(u, v), \text{DRNL})$

- 점수함수가 통째로 학습 대상 — 내적 같은 고정 형태가 없음
- 같은 u 라도 상대 v 에 따라 DRNL 라벨이 달라져 매번 다른 표현으로 계산

한 줄 대비: VGAE는 “노드를 잘 임베딩해두면 링크는 유사도로 따라온다”, SEAL은 “링크는 쌍 주변의 국소 구조 패턴이 결정하니, 그 패턴 자체를 분류기로 배우자”.

VGAE vs SEAL 한눈에 보기

	VGAE	SEAL
관점	그래프 전체 임베딩	후보 링크별 서브그래프 분류
입력 단위	그래프 1개	후보 링크 1개
디코더	내적 $\sigma(\mathbf{z}_u^\top \mathbf{z}_v)$	DGCNN (GCN+SortPool+Conv1D)
노드 라벨	원본 feature만	원본 feature + DRNL one-hot
학습 비용	작음	큼 (서브그래프 전처리)
추론 비용	매우 작음 ($O(d)/\text{쌍}$)	큼 (쌍마다 서브그래프)
표현력	대칭/저차원 한계	휴리스틱 일반화, 풍부
숨은 가정	homophily · 대칭 (내적이 강제)	가정을 데이터에서 학습
새 노드/그래프	transductive — 재학습 필요	inductive — 즉시 추론

실습 노트북 안내

교재의 VGAE·SEAL 구현을 실행 가능한 노트북으로 정리 — `practice/ch10_link_prediction.ipynb`

- **Part 1 — VGAE:** RandomLinkSplit → 인코더 정의 → 300 epoch 학습 → $\hat{A}$ 히트맵, t-SNE 시각화
- **Part 2 — SEAL:** enclosing subgraph 추출 → 타깃 엣지 제거 → DRNL 라벨링 → DGCNN 30 epoch 학습

	책	본 실행 (CPU, 시드 0)
VGAE	AUC 0.88 / AP 0.90	AUC 0.869 / AP 0.872
SEAL	AUC 0.89 / AP 0.90	AUC 0.786 / AP 0.802

SEAL이 책보다 낮은 이유(epoch 수 · 이중 sigmoid 이슈 · 음성 샘플링 시드)는 앞 슬라이드 참조 — 재현성 자체가 메타 교훈.

발제 마무리

한 줄 핵심

링크 예측은 GNN이 자연스럽게 풀 수 없는 **anchored 과제**다. VGAE는 노드 feature에 그 일을 떠넘기고, SEAL은 라벨링으로 직접 해결한다.

- 링크 예측은 쌍에 대한 이진 분류 — 데이터 분할도 엣지 단위, 그래프는 표본
- GAE/VGAE: 노드 임베딩 + 내적 디코더, 확장성이 강점 — 단 내적이 homophily · 대칭을 강제

- SEAL: 서브그래프 분류로 환원, 표현력이 강점 — labeling trick이 1-WL 한계를 깬

다음 장 예고 (Chapter 11): 그래프 생성 모델

Q&A

질문 / 토론

- VGAE의 KL 항을 $1/N$ 로 나누는 이유?
- SEAL에서 enclosing subgraph의 반경 k 를 키우면?
- 산업 환경(수억 노드)에서 SEAL이 적용 가능할까?
- 왜 AUC와 AP를 둘 다 보나? — AUC는 클래스 비율에 둔감해 모델 비교가 안정적, AP는 top- k 사용 시나리오에 가까움
- 사기 탐지처럼 *heterophily* 엣지를 예측해야 한다면 내적 디코더는 어디서 무너지나?
- 타겟 엣지를 서브그래프에서 제거하지 않으면 무슨 일이 일어나나? 학습/테스트 AUC 차이로 진단 가능한가?
- DRNL은 두 노드용 — 세 노드 관계(삼각형 닫힘 예측)로 일반화하려면?